

World Wide Web

System developed for information sharing across internationally dispersed teams, originally aimed at the High Energy Physics community

Stores documents and other web resources, interlinked with each other and accessed via the Internet

World Wide Web

In 1989, Tim Berners Lee invented the three fundamental technologies that are the foundation of the WWW

- HTTP - HyperText Transfer Protocol
- URI - Unique Resource Identifier
- HTML - HyperText Markup Language

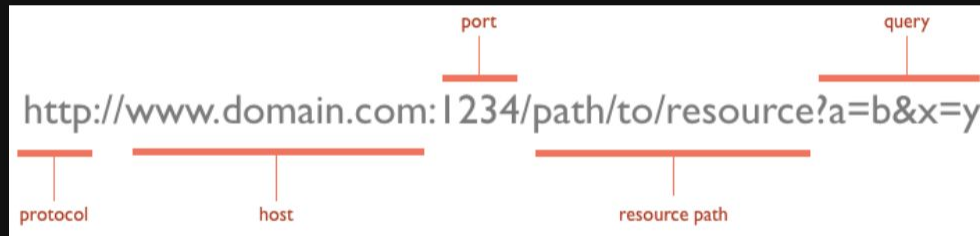
Not only that, he wrote the first web browser and first web server

After that, he created and still leads the World Wide Web Consortium (W3C)

Uniform Resource Locators

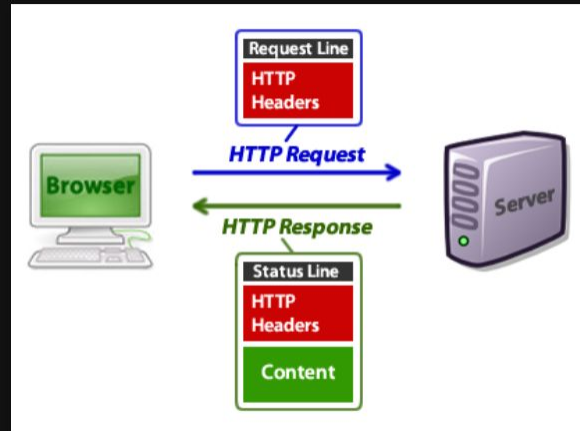
A pointer to a resource on the World Wide Web. Can be something as simple as a file or a directory, or a reference to a more complicated object, such as a query to a database or to a search engine

A Web HTML document can contain a number of URLs, pointing to other Web resources (links)



Hypertext Transfer Protocol

HTTP is a stateless, application-layer protocol for communicating between distributed systems and is the foundation of the modern web. Communication between a host and a client occurs via a request/response pair. The client initiates an HTTP request message, which is serviced through an HTTP response message in return.

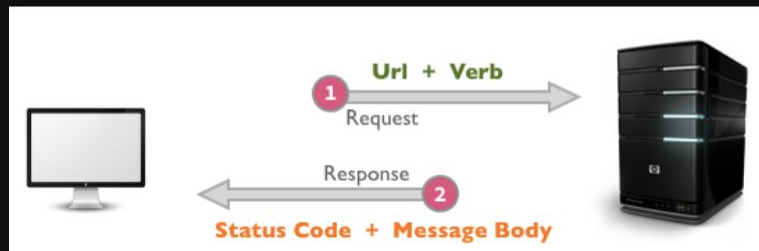


HTTP Verbs

HTTP verbs define the action to perform on a specific web resource

The most popular verbs are:

- GET Fetch an existing resource. The URL contains all the necessary information the server needs to locate and return the resource
- POST Create a new resource. POST requests usually carry a payload that specifies the data for the new resource
- PUT Update an existing resource. The payload may contain the updated data for the resource
- DELETE Delete an existing resource



HTTP Status Codes

With URLs and verbs, the client can initiate requests to the server. In return, the server responds with status codes and message payloads. The status code is important and tells the client how to interpret the server response

Code	Description	Code	Description
200	OK	400	Bad Request
201	Created	401	Unauthorized
202	Accepted	403	Forbidden
301	Moved Permanently	404	Not Found
303	See Other	410	Gone
304	Not Modified	500	Internal Server Error
307	Temporary Redirect	503	Service Unavailable

curl & wget

- command line tools that can download contents from FTP, HTTP and HTTPS
- can send HTTP POST requests
- support HTTP cookies
- designed to work without user interaction, like from within scripts

```
// Dump to screen the contents of the index page at www.google.com  
$ curl http://www.google.com
```

```
// Create a mirror image of GNU WWW site (same directory structure the original has)  
// with only one try per document, saving the log of the activities to a log file  
$ wget -r -t1 http://www.gnu.com -o mirror.log
```

{ Exercise }

Perform manual HTTP GETs for popular websites with
wget/curl

java.net.URL

Immutable object representing an abstraction of a Uniform Resource Locator

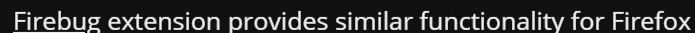
Comes with the capability of retrieving the URL contents and support for proxy servers and multiple protocols, making use of the Strategy design pattern

java.net.URLConnection and its subclasses provide advanced features and control. They provide a way to use sockets in your code without having to know any of the underlying socket details

{ Exercise }

URL source viewer (curl like behaviour)

Built into Google Chrome, they provide web developers with deep access to the internals of the browser and web applications



{ Hands-On }

Inspect network requests with browser dev tools

{ Exercise }

Simple Web Server (support for 1 client only)

Frontend Web Development

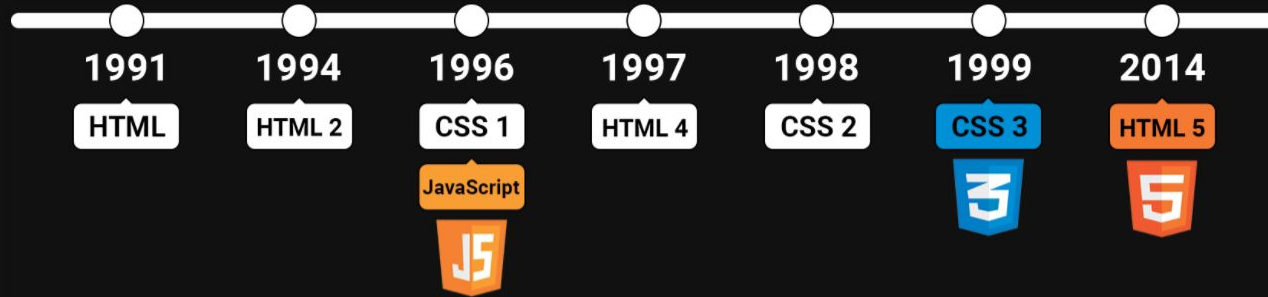
Created by <Academia de Código_>

Frontend

The part of the Web that we can see and interact with



A Little History



HTML



HyperText Markup Language

The most basic building block of the Web.

Made up of **elements** that may be used to:

- Structure the web document into logical sections (navigation, header, content, footer, ...etc)
- Describe the web document's content (text, images, and other kinds of multimedia resources)
- Convey meaning to different pieces of text (title, subtitle, paragraph, list, ..etc)

The diagram illustrates the categorization of HTML elements based on their function. The elements are arranged in a grid, with colors indicating their category:

- Root element** (Green): `html`
- Text-level semantics** (Yellow): `span`, `a`, `rt`, `dfn`, `em`, `i`, `small`, `ins`, `s`, `br`, `p`, `blockquote`, `pre`, `meter`, `select`, `aside`, `h1`, `h2`, `h3`, `h4`, `h5`, `h6`, `section`, `header`, `nav`, `menu`, `command`, `article`, `footer`, `hgroup`, `colgroup`, `caption`, `td`, `th`, `tbody`, `thead`, `tfoot`
- Metadata and scripting** (Blue): `head`, `title`, `meta`, `base`, `link`, `style`, `noscript`, `q`, `var`, `sub`, `mark`, `bdi`, `wbr`, `figcaption`, `ul`, `dt`, `input`, `output`, `keygen`
- Document sections** (Light Blue): `div`, `fieldset`, `form`, `body`, `h1`, `h2`, `h3`, `h4`, `h5`, `h6`
- Grouping content** (Orange): `div`, `fieldset`, `form`, `body`, `h1`, `h2`, `h3`, `h4`, `h5`, `h6`, `section`, `header`, `nav`, `menu`, `command`, `article`, `footer`, `hgroup`, `colgroup`, `caption`, `td`, `th`, `tbody`, `thead`, `tfoot`
- Interactive elements** (Grey): `button`, `progress`, `details`
- Embedding content** (Pink): `img`, `area`, `map`, `embed`, `object`, `param`, `source`, `iframe`, `canvas`, `track`, `audio`, `video`

Anatomy of an HTML element



HTML Elements consist of

- Opening Tag - The name of the element wrapped in opening and closing angle brackets. It states where the element begins or starts to take effect
- Attributes - Modifies the default functionality of an element or provides functionality to certain elements unable to function correctly without them
- Content - The content of the element
- Closing Tag - States where the element ends or stops taking effect

Nesting Elements

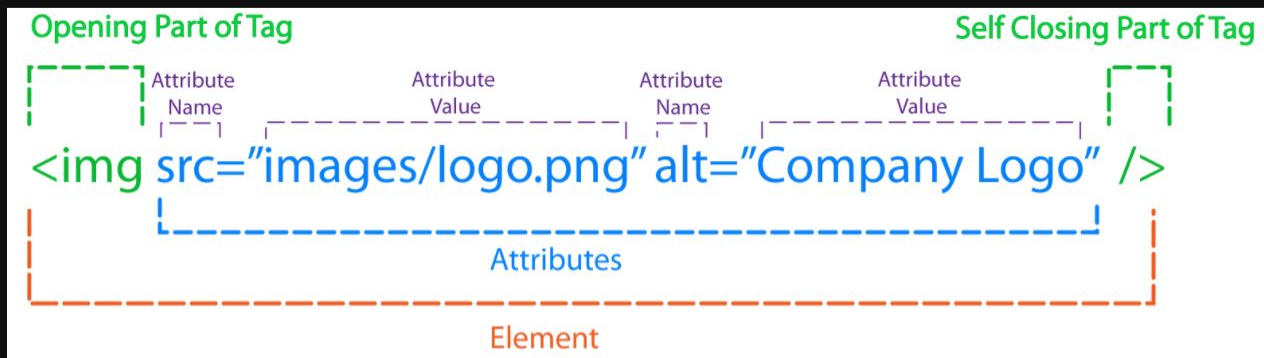
Elements may be nested within other elements.

Element tags need to be *closed* in the same order they were *opened*.

```
<!-- By the way, this is a comment -->  
<!-- this is nested correctly: tags close in the same order as they were opened -->  
<a>Hello, <strong>world</strong>!</a>  
  
<!-- this is nested incorrectly-->  
<a>Hello, <strong>world</a>!</strong>
```

Self Closing Elements

Some HTML elements can use a self-closing tag syntax



here, the forward slash / at the end is optional

HTML Page

- `<!DOCTYPE html>` Special tag to indentify this text file as an HTML document.
- `<html>` Root element for the HTML document that wraps all the other elements
- `<head>` Container for all the meta data associated with the document
- `<meta>` Metadata element specifying the character encoding used
- `<script>` Can either contain javascript code or a link to a javascript file
- `<title>` The title of document
- `<body>` Root element for all the content that will be visible (rendered) in the web page

```
<!-- This is an HTML comment -->

<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>My Special Webpage</title>
    <script src="/index.js"></script>
  </head>
  <body>
    <h1>Hello, <strong>World</strong>!</
  </body>
</html>
```

{ Hands-On }

HTML Hello World

Special Characters

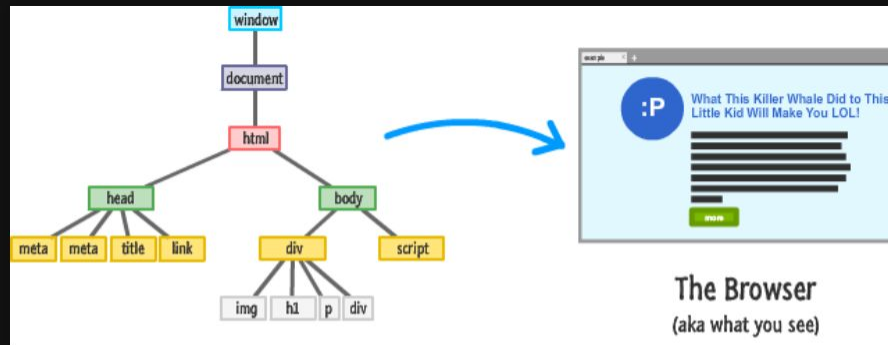
In HTML, the characters `<`, `>`, `"`, `'` and `&` are special characters. They are part of the HTML syntax itself and should not appear as content

Special codes exist that represent these characters and can be used in these circumstances:

Literal character	Character reference equivalent
<code><</code>	<code>&lt;</code>
<code>></code>	<code>&gt;</code>
<code>"</code>	<code>&quot;</code>
<code>'</code>	<code>&apos;</code>
<code>&</code>	<code>&amp;</code>

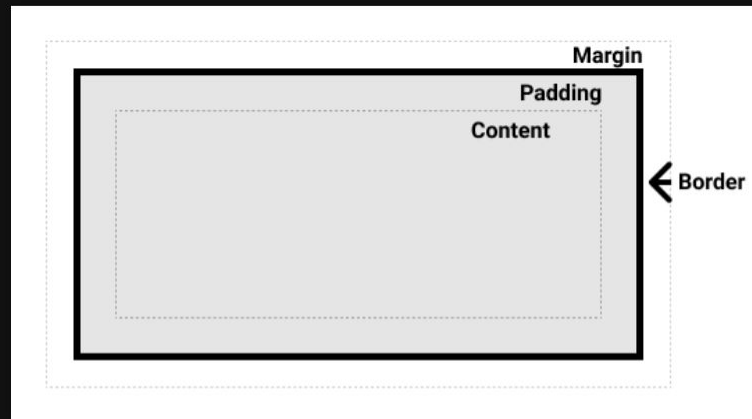
The Document Tree

HTML elements are organized hierarchically on a document tree, and subsequently rendered on screen by the browser



The Box Model

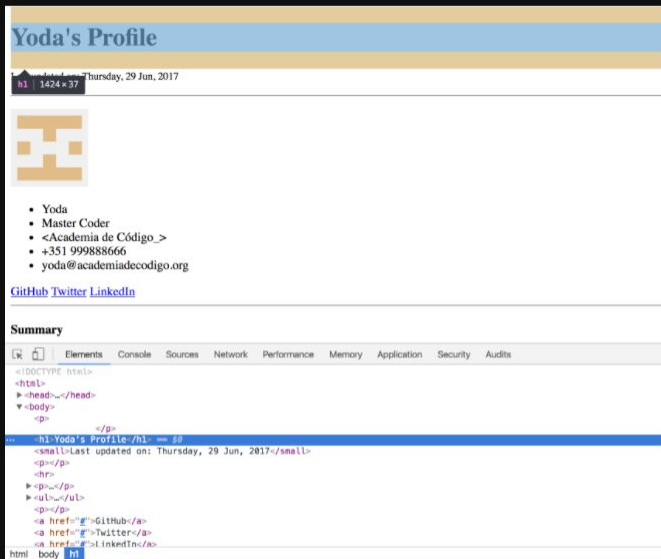
Each element in the HTML document is represented as a rectangular box, with the box's content, padding, border, and margin built up around one another



Each of these box properties can be manipulated for adjusting their format and layout on screen

The Box Model

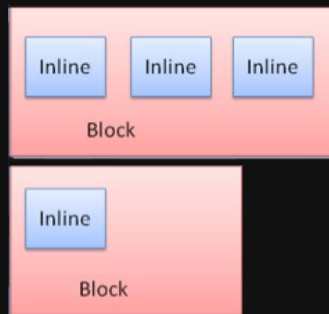
We can visualize the boxes associated with each HTML element on the Chrome Dev Tools



Block vs Inline elements

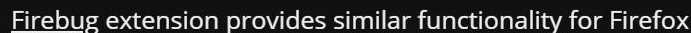
Two important categories of elements in HTML exist:

- Block level elements form their own block. They use all available width and have their own rectangle on the browser window
- Inline level elements do not have their own block. They live inside a containing block



Blocks are formatted vertically one after the other. Inlines are formatted horizontally

Built into Google Chrome, they provide web developers with deep access to the internals of the browser and web applications



{ Hands-On }

Chrome Dev Tools

Where do we start?

One of the most important elements in a web document is... text.

But a title is different from a sub-title or a paragraph, right?



Basic Semantics

HTML provides block elements for paragraphs and headings of different weights.

- Use `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>` to represent headings of different weights (`<h1>` is the biggest!)
- Use `<p>` to represent a text paragraph

Line and Paragraph Breaks

- the `<br>` element represents a line break
- the `<hr>` element represents a break between paragraph-level elements (horizontal rule)

```
<p>  
  Paragraph 1 text  
  <br>  
  some more text  
</p>  
<hr>  
<p>  
  Paragraph 2 text  
  <br>  
  some more text  
</p>
```

Paragraph 1 text
some more text

Paragraph 2 text
some more text

More Semantics

- `<strong>` For **strong** importance text
- `<em>` To give *emphasis* to text
- `<small>` For ^{smaller} size text
- `<code>` For computer code

- Represents an image in the document
- Alternative text can be displayed with the alt attribute

```
  

```

Lists

Unordered `<ul>` lists are used to represent lists of items without a particular order

Ordered `<ol>` lists are used to represent lists of ordered items

Wrap each item inside a **List Item** element `<li>`

```
<ul>
  <li>C</li>
  <li>C++</li>
  <li>Java</li>
  <li>JavaScript</li>
</ul>

<ol>
  <li>Get Up</li>
  <li>Code</li>
  <li>Sleep</li>
</ol>
```

- C
- C++
- Java
- JavaScript

1. Get Up
2. Code
3. Sleep

Tables

```
<table>
  <tr>
    <th>First Header</th>
    <th>Second Header</th>
    <th>Third Header</th>
  </tr>
  <tr>
    <td>First Cell</td>
    <td>Second Cell</td>
    <td>Third Cell</td>
  </tr>
</table>
```

First Header	Second Header	Third Header
--------------	---------------	--------------

First Cell	Second Cell	Third Cell
------------	-------------	------------

<button>

As the name implies, it represents a clickable button.

JavaScript may be used to describe what should happen on user interaction -- but we'll keep that for later.

```
<button>Click Me!</button>
```

Hyperlinks

One of the most fundamental features of the Web.

Allow us to link our documents to any other resource (more documents, images, videos, audio, etc..)

An URL uniquely identifies a resource on the web.

```
http://www.academiadecodigo.org/  
file:///Users/ac/dev/1-frontend-web/1-text/index.html  
images/help.png  
/robots.txt
```

Link up!

- To turn text (or other element) into a link, just wrap it into an anchor tag `<a>`.
- Use the `href` attribute to specify the target URL.
- `target="_self"` opens the document in the same tab (default).
- `target="_blank"` opens the document in a new tab.
- Looks a little bit like **this**.

```
<!-- may be used to link to an external page -->  
<a href="www.academiadecodigo.org" target="_blank">Apply Now!</a>  
<!-- may be used to link to a relative file -->  
<a href="checkout.html">Proceed to checkout!</a>
```

What about an image link?

```
<a href="www.academiadecodigo.org" target="_blank">  
    
</a>
```

Content Structuring

Besides describing the document's content (text, images, etc), HTML provides **block elements** to define its structure.

Despite the fact pages may be more or less complex, they usually share a list of common sections -- structurally speaking.

- Header
- Navigation Bar
- Content
- Sidebar
- Footer



Semantics are important!

You may use CSS visually layout the individual elements to form sections, but the visuals alone won't allow other programs to infer the purpose of certain elements.

Assistive technologies rely on these semantics to provide other means of interacting with the page.

Semantics ARE IMPORTANT!

HTML provides multiple elements to mark up sections of content based on their purpose.

- `<header>`
- `<nav>`
- `<main>`
 - `<article>`
 - `<section>`
 - `<aside>`
- `<footer>`

There are so many more elements to discover!

HTML includes a lot more elements (most of them introduced in version 5) to represent different types of content:

Check them all at [MDN HTML Element Reference](#)

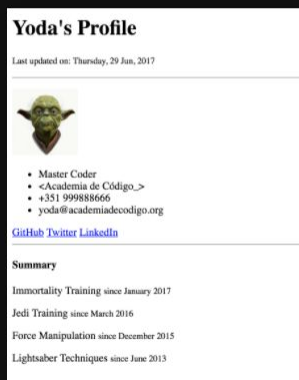
{ Exercise }

Building a personal profile page

Create an index.html file and a resources folder

Add HTML elements of your choice to the web document

Don't forget to use semantic elements to structure the page



CSS



Cascading Style Sheets

Allows the structure of a web page to be separated from its presentation and styling

Specifies how documents are styled, laid out and presented

Web browsers apply CSS rules to HTML elements to affect how they are displayed

Browser Defaults

You've already seen some styling being implemented:

- Headings are rendered in bold, with a lot of space around them
- Unordered lists are indented and rendered with a preceding symbol (like this one)
- Hyperlinks are underlined and rendered in blue or purple
- Button text is wrapped in a gray rectangle

Bring Your Own Styling

You may extend browser default styles using **CSS rules**.

Rules specify a set of styles to be applied to a group of elements.

A **rule** looks like this:

```
h1 {  
  color: red;  
  font-size: 12px;  
}
```

- **h1** is the **selector**: targets a group of elements
- **declarations** describe styles to be applied
- **declarations** are written inside brackets as `property:value` pairs

Documents with Style

The `<link>` element links a web document to an external resource.

It may be used to apply CSS rules defined in a CSS file.

```
<html>
  <head>
    <!-- link to a CSS file. all rules will be applied on this document -->
    <link rel="stylesheet" href="styles.css">
    <!-- link to another CSS file. all rules will be applied on this document -->
    <link rel="stylesheet" href="more-styles.css">
  </head>
  <body>
  </body>
</html>
```

- `rel` describes the relationship between the web document and the resource
- `href` specifies the resource's URL

Classes

Styling elements based on their names is limiting. What if we wanted to have two paragraphs of different colors?

Classes to the rescue!

- Class names may be added to HTML elements through the `class` attribute.
- **Selectors** are able to target elements by their class names using `.className` syntax.

```
<p class="error" >I am an error.<p>  
<p class="success">Success!<p>  
<h1 class="big success">VERY SUCCESSFUL!<h1>  
<p>I'm just a paragraph.<p>
```

```
.error {  
  color: red;  
}  
  
.success {  
  color: blue;  
}  
  
.big {  
  font-size: 50px;  
}
```

ID, please.

Selectors are also able to target HTML elements by their `id` attribute.

`id` attribute values should be unique across all document.

```
<p id="special-secret">I don't want to be seen.</p>
```

```
#special-secret {  
  opacity: 0;  
}
```

{ Hands-On }

Explore classes and IDs

Advanced Selections

It's possible to combine **selectors** using **combinators**, which allows:

- applying same styles to multiple groups of elements
- targeting elements based on their location

```
/* this is a CSS comment by the way */  
/* all paragraphs and all elements with class "success" will be blue */  
p, .success {  
    color: blue;  
}  
  
/* all paragraphs with the "warning" class will be red */  
p.warning {  
    color: red;  
}  
  
/* all elements with "blue" class that are descendents of an element with "bright" class with  
.bright .blue {  
    color: lightblue;  
}  
  
/* all sections nested inside an element with id "main-area" will have a border */  
#main-area > section {  
    border: 2px solid blue;  
}
```

Level up your selecting game!

Get comfortable with **selectors** with this [CSS Selector Tutorial](#).

You can use [CSS Selectors Reference](#) for extensive documentation.

So many styles to choose from

There are three core concepts in CSS that define which styles get applied to an element.

- Cascading
- Specificity
- Inheritance

To truly master CSS, we need to understand these three concepts!

Cascading in CSS

Cascading is a core concept in **CSS**.

Stylesheets **cascade**. The order of the rules matter: the last one will have precedence over previous ones.

If a document is linked to multiple stylesheets, the last ones will have precedence over the previous ones.

```
h1 {  
  font-size: 10px;  
}  
  
h1 {  
  font-size: 12px;  
}
```

<h1> font-size will be 12px!

Specificity

Another core concept in **CSS**.

Defines which styles will be applied in case of conflicting rules.

Specificity states that more specific selectors will have precedence over broader ones.

- element name
- class
- id

```
img {  
  border-radius: 0px;  
}  
  
.round-corners {  
  border-radius: 10px;  
}  
  
#ball {  
  border-radius: 100%;  
}
```

What will be the border-radius of `<img id="ball" class="round-corners"/>`?

Inheritance

There are two kinds of properties:

- **Inherited** properties
- **Non-inherited** properties

Children elements **inherit** parent's **inherited** properties.

```
<header>
  <h1>I'll be cool and centered</h1>
  <h4>So am I</h4>
  <p>Me too!</p>
</header>
```

```
header {
  text-align: center;
  color: blue;
  border: 1px solid red;
}
```

{ Hands-On }

Use Chrome DevTools to debug styles being to elements.

Element Boxes

HTML elements wrap content inside a box.

Web browsers render content and its **box** using a set of rules called **CSS Box Model**.

With CSS, it's possible to define how the content is positioned inside its own **box** and how all **boxes** are layed out in the document.

CSS Box Model

Specification that defines how element boxes are rendered



- Content box: the area where your content is displayed. Can be sized using `width` and `height` properties.
- Padding box: white space around content. Can be sized using padding related properties.
- Border box: border box wraps content and padding. Can be controlled using border related properties.
- Margin box: white space around content, padding and border box. can be controlled using margin related properties

Boxes, boxes, and more boxes

CSS broadly defines two types of boxes.

They mostly differ in how they handle parent's available space.

- **Block boxes**
- **Inline boxes**

The `display` property defines which kind of box surrounds the content.

Block Boxes

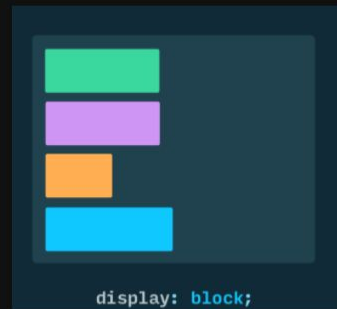
Break onto a new line, stacking on top of each other.

Fill available parent space (in the writing direction) by default.

Respect width and height properties.

If width is not defined, they are as wide as their parent.

Paddings, margins and borders push other elements away.



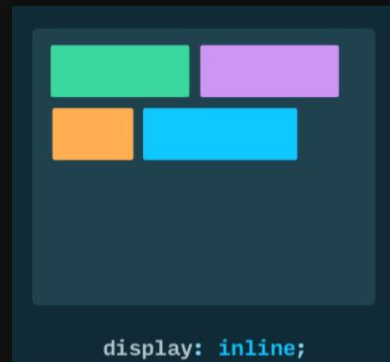
Inline Boxes

Do not break onto a new line.

Width and height properties do not apply.

Element size is always the sum of **content size, padding, border width** and **margin**.

Horizontal paddings push other inline elements away.



Inline-block Boxes

There's a special kind of box that is something in-between.

It's like an **inline** box but width and height may be set.

If undefined, box size will be the same as an inline box.

```
display: inline-block;
```

Generic Elements

HTML provides two generic elements for content wrapping.

These elements hold **no semantic meaning**, unlike `<article>`, `<section>` or `<h1>`.

- `<div>` to wrap content in a **block** box
- `<span>` to wrap content in an **inline** box

{ Hands-On }

Experiments with box model properties

Content overflow

So, everything in CSS is a box, right?

What happens if there is too much content to fit in the box?

Overflow

Good guy CSS

By default, CSS tries to avoid hiding data.

If the box is not big enough, content will **overflow** the box limits.

```
<div class="slim">
  <p>hellooooooooooooooooo world!</p>
</div>
<div class="short">
  <p>hello</p>
  <p>...</p>
  <p>...</p>
  <p>...</p>
  <p>world!</p>
</div>
```

```
.slim {
  width: 15px;
}

.short {
  height: 20px;
}
```

overflow property

Allows changing how browser handles overflowing content.

It's possible to hide overflowing content completely, or enable scrollbars to scroll through.

```
.slim {  
  width: 15px;  
  overflow: hidden;  
}  
  
.short {  
  height: 20px;  
  overflow: scroll;  
}
```

{ Hands-On }

Experiments with overflow property

{ Exercise }

Create a visual representation of the CSS Box Model using HTML and CSS.

CSS Layout

HTML lays out elements using the rules of **block** and **inline** boxes.

A well structured HTML document is very important, but the **normal flow** is not sufficient for complex layouts.

CSS provides a set of tools to change how the browser positions elements on the document.

Basic document flow

I am a basic block level element. My adjacent block level elements sit on new lines below me.

By default we span 100% of the width of our parent element, and we are as tall as our child content. Our total width and height is our content + padding + border width/height.

We are separated by our margins. Because of margin collapsing, we are separated by the width of one of our margins, not both.

inline elements like this one and this one sit on the same line as one another, and adjacent text nodes, if there is space on the same line. Overflowing inline elements will wrap onto a new line if possible (like this one containing text), or just go on to a new line if not, much like this image will do:



display property

As we've seen before, the `display` property is able to change the type of the element's box.

```
display: block;  
display: inline;  
display: inline-block;
```

{ Hands-On }

Explore basic document flow and display property.

display property superpowers

display property also provides two possible values that change how child elements are layed out.

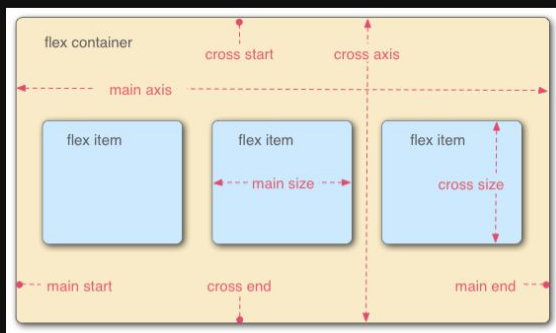
These are extremely useful when creating complex layouts.

Although children's rendering flow is changed, the element itself is treated as a **block**.

```
display: flex;  
display: grid;
```

Flexible Box Layout (aka Flexbox)

Easily lays out elements in a single dimension (row or column).



```
<div class="parent">  
  <div class="child">one</div>  
  <div class="important child">two</div>  
  <div class="child">three</div>  
</div>
```

```
.parent {  
  outline: 1px solid blue;  
  display: flex;  
}  
  
.child {  
  outline: 1px dotted red;  
}
```

Flexbox children flow

Flexbox provides extra properties to tune how children are layed out.

```
.parent {  
  flex-direction: column;  
  justify-content: stretch; /* changes main axis flow */  
  align-items: center; /* changes cross axis flow */  
}
```

Flexbox children flow

Flex container children may define how much available space they fill in the main axis through the `flex` property.

The value may be any positive number and represents a relative size.

```
.child {  
  flex: 1;  
  outline: 1px solid red;  
}  
  
.important {  
  flex: 2;  
}
```

{ Hands-On }

Flexbox Flow

{ Exercise }

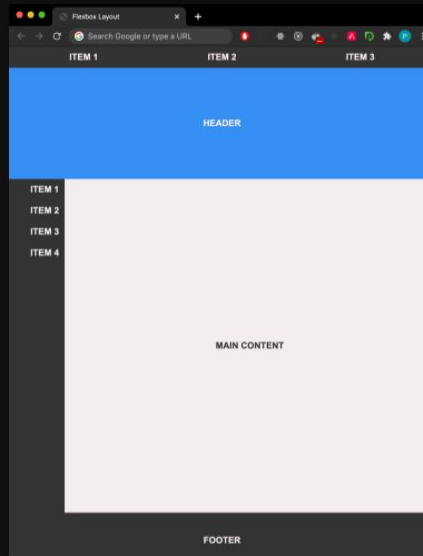
Learn Flexbox with **Flexbox Froggy**!

Useful resources:

- **Flexbox cheatsheet**
- **Flexbox in-depth with guides**

{ Exercise }

Page layout with Flexbox



```
<nav class="navbar">
    <ul>
        <li>ITEM 1</li>
        <li>ITEM 2</li>
        <li>ITEM 3</li>
    </ul>
</nav>
<header>HEADER</header>
<main>
    <aside>
        <nav class="sidebar">
            <ul>
                <li>ITEM 1</li>
                <li>ITEM 2</li>
                <li>ITEM 3</li>
                <li>ITEM 4</li>
            </ul>
        </nav>
    </aside>
    <div class="main-content">MAIN CONTENT</div>
</main>
```

```
.navbar {  
  height: 40px;  
}  
  
header {  
  height: 200px;  
}  
  
.main-content {  
  height: 200px;  
}  
  
.sidebar {  
  width: 100px;  
}  
  
footer {  
  height: 100px;  
}
```

Element positioning

Positioning allows moving an element from where it would be placed in normal document flow.

Suited for managing and fine-tuning a specific element's position.

There are 5 types of positioning:

- **Static** positioning
- **Relative** positioning
- **Absolute** positioning
- **Fixed** positioning
- **Sticky** positioning

position property

Element positioning is **static** by default, but may be changed through the position property.

```
position: static; /* default value */  
position: relative;  
position: absolute;  
position: fixed;  
position: sticky;
```

Static positioning

The element is positioned where it should be in the document flow.

```
<div class="parent">  
  <div class="box" id="one">One</div>  
  <div class="box" id="two">Two</div>  
  <div class="box" id="three">Three</div>  
  <div class="box" id="four">Four</div>  
</div>
```



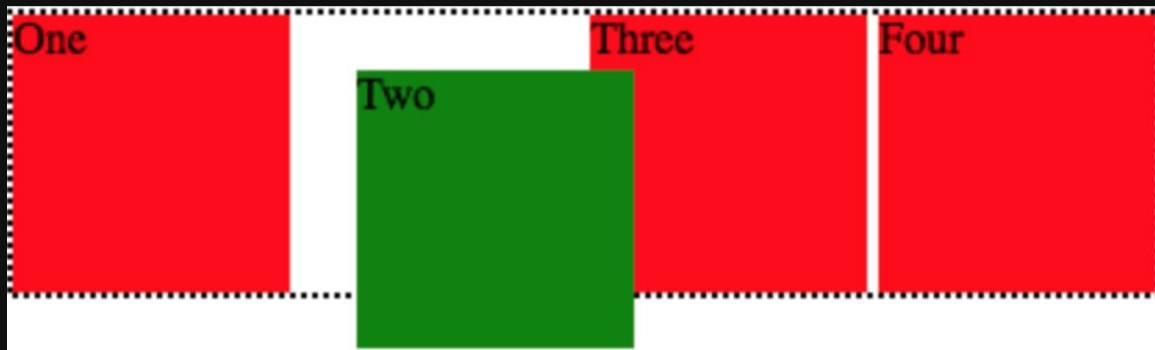
```
.parent {  
  border: 2px black dotted;  
  display: inline-block;  
}  
  
.box {  
  display: inline-block;  
  background: red;  
  width: 100px;  
  height: 100px;  
}  
  
#two {  
  background: green;  
}
```

Relative positioning

Allows offsetting an element position.

This may be done with top, left, right and bottom properties.

```
#two {  
  top: 20px;  
  left: 20px;  
  background: green;  
  position: relative;  
}
```



Absolute Positioning

Removes an element from the document flow.

Element is positioned relative to the nearest positioned parent.

```
.parent {  
  border: 2px black dotted;  
  display: inline-block;  
  position: relative;  
}  
  
#two {  
  top: 20px;  
  left: 20px;  
  background: green;  
  position: absolute;  
}
```

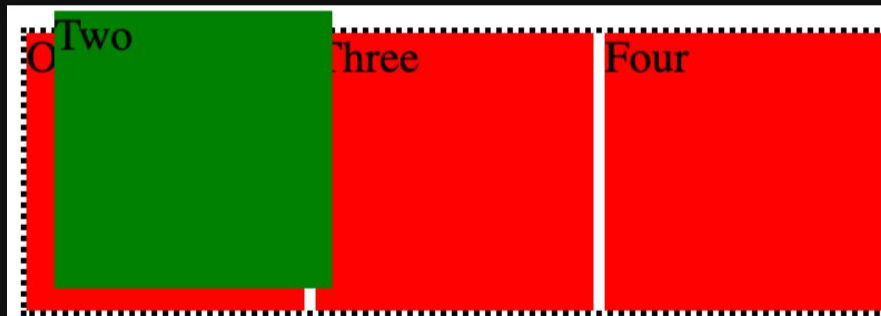


Fixed Positioning

Removes an element from the document flow.

Element is positioned relatively to the viewport, which makes it look **fixed** in place.

```
#two {  
  top: 20px;  
  left: 20px;  
  background: green;  
  position: fixed;  
}
```



Sticky Positioning

Statically positioned item until scrolled to the desired viewport offset.

Once it hits the desired offset, it will act as a **fixed** position element.

Older browsers do not support this value.

```
#two {  
  top: 20px;  
  left: 20px;  
  background: green;  
  position: sticky;  
}
```

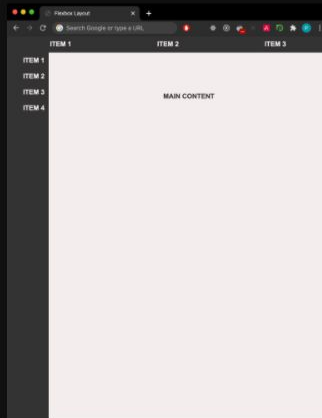
Let's try it ourselves, shall we?

{ Hands-On }

Positioning elements!

{ Exercise }

Update your Flexbox Layout page to have a fixed navigation bar and a sticky sidebar!



```
#main-content {  
  height: 2000px;  
}
```

Google Fonts

Extensive catalog of open source designer web fonts

Add a stylesheet link to request the desired web fonts

```
<link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Tangerine">
```

Style the element with the requested web font

```
p {  
  font-family: 'Tangerine', serif;  
  font-size: 48px;  
  text-shadow: 4px 4px 4px #aaa;  
}
```

Making the Web Beautiful!

Font Awesome



Scalable vector icons using a single font and css

The font awesome css can be loaded using a cdn link

```
<link href="http://maxcdn.bootstrapcdn.com/font-awesome/4.2.0/css/font-awesome.min.css" rel="st
```

Insert icons easily with the `i` element

```
<!-- github icon -->  
<i class="fa fa-github"></i>
```

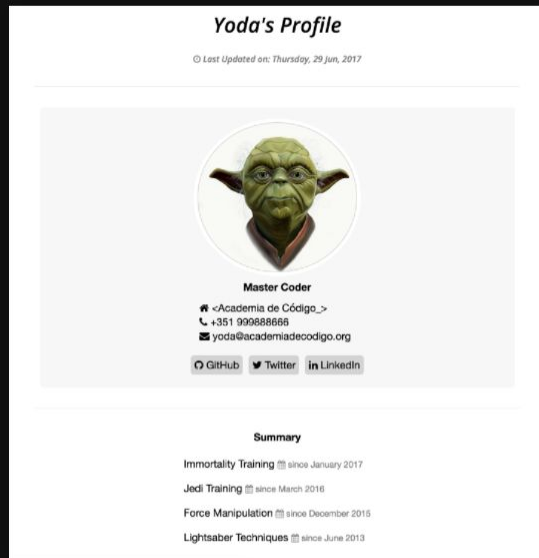
Mozilla Developer Network



The MDN is the official website for development documentation of web standards, explore it to take full advantage of the web's capabilities

{ Exercise }

Add some styling to your profile page



Size does matter

The advent of the web has brought browsers to an ever-growing number of devices with different screen sizes.

This brought up a question to developers and designers everywhere...

How can my web site look good on them all?

Responsive Web Design

A set of practices that allows web pages to alter their layout and appearance to suit different screen sizes.

Initially based on complex legacy CSS layout techniques and **media queries**.

CSS has since evolved, providing tools to develop *responsively* easily.

Flexbox is one of those tools! Did you notice how responsive your Flexbox Layout was?

Media Queries

Available around 2010, it's one of the most important CSS features to truly allow **Responsive Web Design**.

Media Queries allow developers to apply different CSS styles depending on viewing device's screen characteristics.

```
@media screen and (min-width: 800px) {  
  /* these styles will only be applied if it's a screen that's at least 800px wide */  
  .container {  
    margin: 1em 2em;  
  }  
}
```

Mobile First Development

Web is being increasibly accessed through mobile devices.

Developers have found it's easier to develop a web page targeting small screens, and then use media queries to make it look good on bigger ones.

{ Hands-On }

Mobile First Development with Chrome DevTools!

Relative Values

CSS allows usage of relative values in most size-related properties.

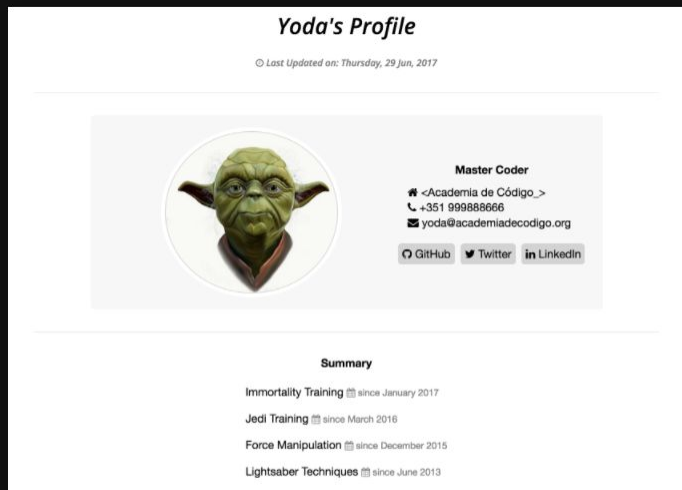
- **%** - percentage of parent's same property value (in most cases). useful for relative `width` and `height`.
- **em** - size relative to parent's `font-size`. useful for relative font sizes.
- **rem** - size relative to root `font-size`. useful for relative font sizes.

{ Hands-On }

Relative font sizes

{ Exercise }

Improve the layout and positioning of the elements in your profile page

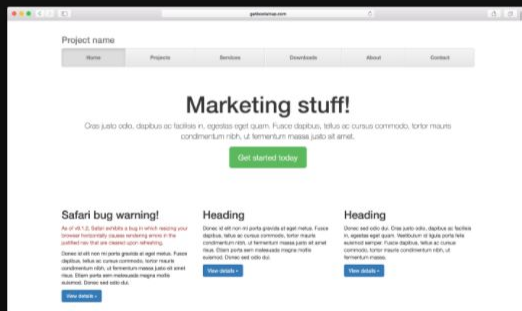


Bootstrap



The most popular front-end framework

- HTML/CSS/JavaScript framework
- Mobile first responsive layouts and grids
- Tools for typography, forms, buttons, navigation and other interface components
- Developed at Twitter but open sourced since 2011



Using Bootstrap

Bootstrap can be downloaded from their website getbootstrap.com, or alternatively used via a CDN:

```
<!-- Bootstrap depends on the jQuery JavaScript library -->
<script src="https://code.jquery.com/jquery-latest.js"></script>

<!-- Latest compiled and minified CSS -->
<link rel="stylesheet"
      href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css">

<!-- Latest compiled and minified JavaScript -->
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js">
</script>
```

To support small screens, the responsive design html5 meta tag should be used

```
<!-- tells the browser to scale the page down so it fits the screen -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Containers

Bootstrap requires a containing element to wrap the site contents

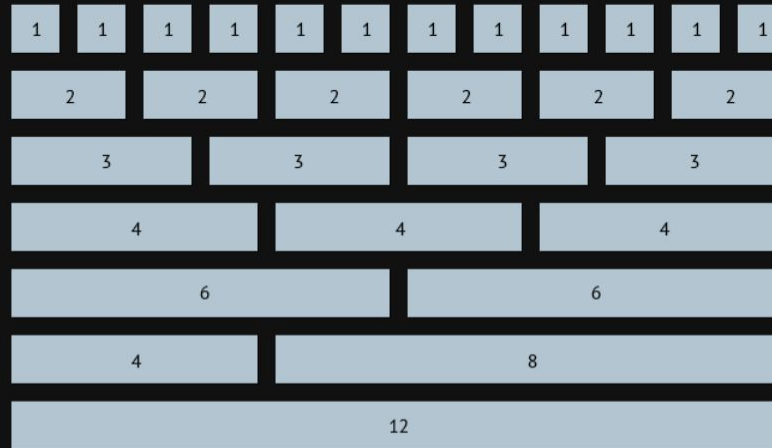
Two types of containers are available

- Responsive fixed width container
- Full-width fluid container

```
<div class="container">  
  <!-- has one fixed width for each screen size -->  
</div>  
  
<div class="container-fluid">  
  <!-- constantly resize as browser width changes -->  
</div>
```

Grid System

Is a responsive mobile-first grid system that scales up to 12 columns as the viewport size increases



Content can span a different number of columns

Grid Classes

Screen sizes are divided into 4 types with different css prefixes

- col-xs-* - extra small devices (phones, less than 768px, bootstrap default)
- col-sm-* - small devices (tablets, 768px and up)
- col-md-* - medium devices (desktops, 992px and up)
- col-lg-* - large devices (large desktops, 1200px and up)

```
<!-- grid needs to be wrapped in a container div -->
<div class="container-fluid">
  <!-- every row must have 12 columns -->
  <div class="row">
    <div class="col-sm-6 col-lg-4">
      <!-- content -->
    </div>
    <div class="col-sm-6 col-lg-4">
      <!-- content -->
    </div>
    <div class="col-sm-6 col-lg-4">
      <!-- content wraps to new row if number of columns > 12 -->
    </div>
  </div>
</div>
```

one column on phones (default), two columns on tablets/desktops (size cascades up) and 3 on large desktops

Bootstrap visibility

- Bootstrap allows you to set the visibility of an element depending on screen size
- An element can either be shown or hidden with the help of visible and hidden class prefixes

```
<div class="container-fluid">  
  <div class="row">  
    <div class="hidden-xs">hide this on phones</div>  
    <div class="visible-lg">show this on large desktops</div>  
  </div>  
</div>
```

Typography

By default, Bootstrap offers great looking fonts and size ratios of headers and paragraphs

[Bootstrap](#) [Overview](#) [About](#) [Grid](#) [Layouts](#) [Typography](#) [Tables](#) [Forms](#) [Navigation](#) [Alerts](#) [Popovers](#) [Less](#)

Typography

Headings, paragraphs, lists, and other inline type elements

Headings and copy

A standard typographic hierarchy for structuring your webpages.

Misc. elements

Using emphasis, addresses, & abbreviations

`<strong>` `<em>` `<address>`
`<abbr>`

h1. Heading 1

h2. Heading 2

h3. Heading 3

h4. Heading 4

h5. Heading 5

H6. HEADING 6

When to use

Emphasis tags (`<strong>` and `<em>`) should be used to indicate additional importance or emphasis of a word or phrase relative to its surrounding copy. Use `<strong>` for importance and `<em>` for stress emphasis.

Emphasis in a paragraph

Fusce dapibus, tellus ac cursus commodo, tortor mauris condimentum nibh, ut fermentum massa justo sit amet risus. Maecenas faucibus mollis interdum. Nulla vitae elit libero, a pharetra augue.

Note: It's still okay to use `<b>` and `<i>` tags in HTML5, but they don't come with inherent styles anymore. `<b>` is meant to highlight words or phrases without conveying additional importance, while `<i>` is mostly for voice, technical terms, etc.

Example paragraph

Nullam quis risus eget urna mollis ornare vel eu leo. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Nullam id dolor id nibh ultricies vehicula ut id elit.

Example heading

Has sub-heading...

Tables

Bootstrap provides various table styles and functionalities

ID	Name	Created at	Actions	
1	Settlers of Catan	2012-03-04 11:07:43 UTC	Edit	Destroy
2	Agricola	2012-03-04 11:07:43 UTC	Edit	Destroy
3	7 Wonders	2012-03-04 11:07:43 UTC	Edit	Destroy
4	Stone Age	2012-03-04 11:07:43 UTC	Edit	Destroy
5	Dominion	2012-03-04 11:07:43 UTC	Edit	Destroy
6	Goban	2012-03-04 11:07:44 UTC	Edit	Destroy

Forms

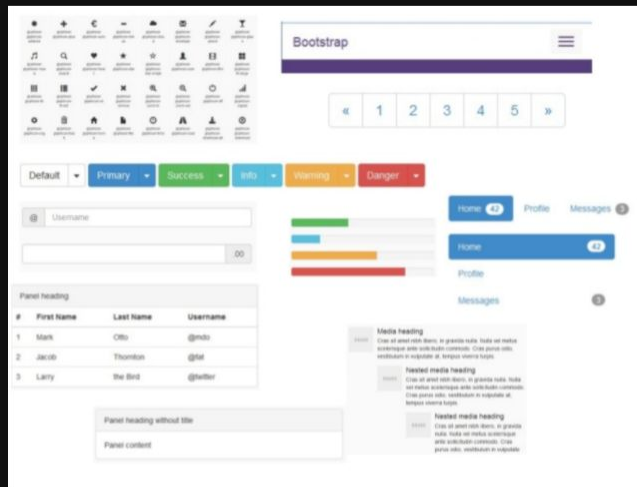
Bootstrap delivers various form control styles, layout features and custom made components for setting up a variety of forms

The image shows a Bootstrap form layout with the following elements:

- Alphabets:** A text input with placeholder "Type something". Below it, a red error message says "This value is required."
- Equal To:** Two text inputs. The first has placeholder "Password" and a red error message "This value is required." below it. The second has placeholder "Re-Type Password" and a red error message "This value is required." below it.
- E-Mail:** A text input with placeholder "Enter a valid e-mail" and a red error message "This value is required." below it.
- URL:** A text input with placeholder "URL" and a red error message "This value is required." below it.
- Number:** A text input with placeholder "Enter only numbers" and a red error message "This value is required." below it.
- Alphanumeric:** A text input with placeholder "Enter alphanumeric value" and a red error message "This value is required." below it.
- Select Box:** A dropdown menu with the selected option "- Select -". Below it, a red error message says "This value is required."
- Textarea:** A text area with a red error message "This value is required." below it.
- Buttons:** At the bottom, there are two buttons: "Submit" (blue) and "Cancel" (white with blue border).

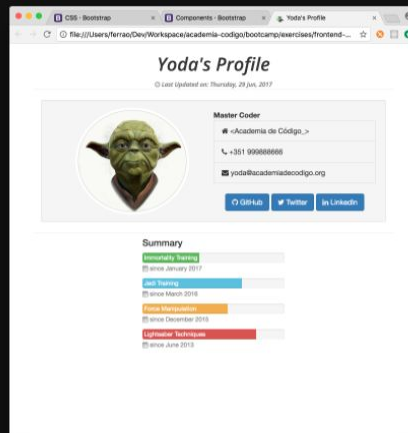
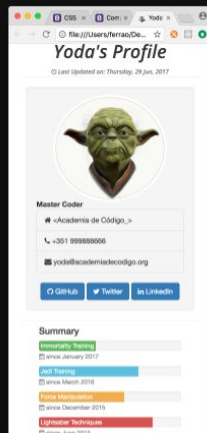
Components

Many reusable components are available, proving iconography, dropdowns, input groups, navigation, alerts and much more



{ Exercise }

Improve the styling and responsiveness of the profile page with bootstrap



{ Extra Exercise }

Learn CSS Grid with [Grid Garden](#).